On-Device Unit Conversion and File Optimization in an Android Application: A Source-Level Study

Ikbal Fadilah
Causyne
ikbal@causyne.com
ORCID: 0009-0008-4853-135X

Abstract

An Android interface groups unit conversion, PDF compression, and image compression under a common utility label. The grouping is useful to a person choosing a tool, but the three implementations have different correctness conditions. I inspect the converter's fixed factors and temperature equations, the image path's bitmap decoding and JPEG/WebP re-encoding, and the PDF path's object-stream rewriting. Several limits are visible without a benchmark: displayed conversion values are rounded to six decimal places; image sampling is not a hard bound on either decoded dimension; a nominally lossless PDF mode still removes metadata; and both compressors can return a completed output without proving that it is smaller or semantically unchanged. The report supplies worked source-level traces and a validation design separating numerical error, visual fidelity, PDF integrity, device resources, and the exact meaning of local processing. It reports no measured compression ratio, latency, memory peak, or end-to-end offline trace. Its contribution is a reproducible account of what the current code does and which observations are needed before broader claims are made.

Keywords: Android, unit conversion, image compression, PDF rewriting, local computing, source audit

1 Introduction

The label "Logic Engine" can suggest a single computational core. In the inspected application it is an entry point to two tabs. The File Engine tab invokes PDF and image tools; the Unit Converter tab presents numerical conversions. A second tools route can also open the file screen. This matters for research because the tabs do not share an algorithm, a correctness metric, or even the same kind of input. A kilogram-to-pound conversion is judged against a stated factor. A recompressed photograph needs a size–quality comparison. A rewritten PDF must remain parseable and retain the properties the user cares about. One "success" count across all three would hide those differences.

The question here is deliberately narrower than whether the application is a mature offline office suite. What transformations can be established from the code, which changes are made to the user's data, and what tests would distinguish a useful optimization from an unsafe or misleading one? The source reveals enough to answer the first two questions at method level. It does not supply a controlled device experiment, an independent image-quality assessment, or a corpus of PDFs checked in multiple readers. I therefore use *source observation* for statements visible in code and *proposed test* for measurements not yet performed.

Naming deserves precision. The author and research identity in this report are Causyne. The inspected Android package and some on-device output strings still use the legacy Vanexa name because the installed application has not been renamed in its distribution channel. These names identify different things: author affiliation and current product branding on one side, existing application artifacts on the other. The paper changes no Android source or Play listing. It also does not treat a source-level local operation as proof that every surrounding workflow is offline.

The manuscript is an implementation study, not a measured performance paper. A source audit can find missing guards and derive counterexamples. It cannot estimate how often they occur in a population of files. To keep those forms of evidence separate, each worked example below states its input and follows the actual branches of the implementation. The final sections define the files, devices, and independent checks needed for an empirical revision.

2 Method and implementation boundary

The inspection follows the user-visible path rather than starting from class names alone. The home-card action opens `ConverterActivity`. Its first tab renders `PdfToolScreen`, which presents PDF and image compression; the second renders `UnitConverterScreen`. The converter screen calls `UnitConverterViewModel`, which dispatches to `UnitConverterUseCase`. The file screen obtains user-selected URIs with Android's document picker and calls either `PdfTools.compressPdf` or `ImageTools.compressImage`. A success callback stores an app-cache path; a later "Save to Device" action copies that file to a user-visible location. A separate route in `ToolsActivity` points to the same file screen. These call sites establish availability in the inspected source, not a claim about every distributed build.

The Android document picker can return URIs from local or cloud-backed providers [2]. Processing a stream on the device does not show where a provider obtained the bytes. The relevant offline boundary begins once the input stream is available to the method. A stronger end-to-end claim would require a test with a locally stored file, a disabled network, and observation of attempted connections by the application and provider. The method calls examined here contain no explicit upload of the image or PDF being compressed, but that negative source observation is not a network trace.

The following table separates the routes and their immediate outputs. It is a map of code paths, not a scorecard of achieved quality.

Table 1: Inspected paths under the utility interface. "Observed" describes code behavior, not a completed device trial.

Path	Input and method	Immediate result
Unit converter	Decimal text; category-specific factors or an affine temperature formula.	Six-decimal display string in screen state.
Image tool	Picker URI; bitmap bounds probe, optional subsampling, JPEG or WebP encoding.	Encoded file in app cache, then optional save.
PDF tool	Picker URI; PDFBox-Android object traversal and document save.	Rewritten PDF in app cache, then optional save.

The source set inspected for this report consists of the converter model, use case, and view model; the image and PDF utility objects; and their immediate UI call sites. The PDF utility also contains a DOCX-to-PDF method, but the corresponding card is disabled in the inspected file screen. That conversion is studied separately in the document-workflow manuscript. Keeping it outside the present experiment prevents a source method that users cannot currently select from inflating the scope of the File Engine.

3 Unit conversion: factors, display, and domain

3.1 Base-unit strategy

Seven converter categories use a base-unit factor. For a source unit with factor a and a target unit with factor b, the code computes

$$y = x\frac{a}{b}. \tag{1}$$

The base is kilogram for mass, meter for length, liter for volume, square meter for area, meter per second for speed, second for time, and byte for data. This arrangement makes a conversion pairwise without storing every ordered pair of units. It is simple to audit: a wrong result can come from one of two factors, arithmetic, or formatting. The constants are stored as `Double` values in enum definitions rather than looked up from a remote service.

Temperature takes a separate path. The code converts the input to Celsius and then converts Celsius to the selected output scale. For Fahrenheit and Kelvin its first step is $(x-32)5/9$ and $x-273.15$, respectively. Its second step is $c(9/5)+32$ or $c+273.15$. This is appropriate to an affine scale: substituting a temperature factor into Eq. 1 would fail at the freezing point. The source does not impose a physical-domain check, however. It will calculate a Kelvin value below zero if given a sufficiently negative Celsius input. Such a result is arithmetically consistent with the formula but not a valid thermodynamic temperature.

Published unit guidance distinguishes exact definitions, conventional approximations, and rounding of reported values [1]. The source uses some rounded decimal factors. For example, one pound is represented as `0.453592` kilograms and one mile as `1609.34` meters. An output calculated with these constants is not "extreme precision" merely because it is displayed with six decimal places. Numerical evaluation must compare against an identified reference factor and report

both the factor's representation error and the display error. The paper does not assign a precision class to the present implementation.

3.2 A worked conversion and the formatting boundary

For the code's kilogram-to-pound path, one kilogram gives $1/0.453592 \approx 2.204625$ pounds before six-place display rounding. The reverse path multiplies by the same stored factor. A round trip can therefore return close to one kilogram even when the factor differs from a higher-precision reference. Round-trip agreement alone is a weak accuracy test if both directions share one erroneous constant. A stronger test compares each direction with an independent standard and covers magnitudes from very small to very large.

The view model parses the entered text using `toDoubleOrNull()`, dispatches by category, and formats the result with `String.format("%.6f", result)` before trimming trailing zeroes and a final decimal point. This is a presentation decision with observable consequences. A value smaller than half a micro-unit may display as zero even though the underlying `Double` is nonzero. The display also uses the device's formatting locale, while `toDoubleOrNull()` expects a particular decimal syntax. A locale using a comma as decimal separator needs a dedicated input–output test; source inspection alone does not establish how every keyboard and locale combination behaves. There is no visible rejection of `NaN` or infinite values in the conversion use case.

The time category defines a month as 2,592,000 seconds and a year as 31,536,000 seconds, equivalent to 30 and 365 days. These are useful fixed conventions, not calendar-aware durations. An actual calendar month varies in length, and a leap year differs from the fixed year. The data category uses powers of 1024 while labels are `KB`, `MB`, and so on. Under binary-prefix conventions these factors correspond to KiB and MiB; a user reading decimal SI prefixes might expect powers of 1000. A study of correctness must therefore record the convention disclosed to the user, not merely compare the arithmetic to the enum constants.

3.3 Testability and what was not measured

The Android repository contains unit tests for representative converter paths: mass, length, temperature, volume, area, speed, time, data, and identity. The full Android test tree did not compile during this audit because unrelated test sources failed. I therefore copied the unchanged `UnitConverter.kt`, `UnitConverterUseCase.kt`, and their original JUnit tests into an isolated Kotlin/JVM module. With JDK 17 and Gradle 8.11.1, all 18 tests passed with no failures. This verifies those tested numerical cases in the isolated module, not the Android user interface, every ordered unit pair, or device behavior. The released source manifest records hashes so readers can compare the copies with the inspected Android files.

For a later experiment, table-driven tests can generate every ordered unit pair, including identity, zero, negative values where meaningful, and scale extremes. Round-trip error, absolute error, and relative error answer different questions. An error near zero should be reported in absolute units because a relative denominator near zero is unstable. A display test must compare the formatted string as well as the `Double` result. It should set the locale explicitly, then feed that displayed text back through the input parser to reveal a locale mismatch if one exists. These are proposed tests, not completed measurements.

4 Image re-encoding: memory and fidelity

4.1 What the path actually does

`ImageTools.compressImage` first queries a provider's size metadata and rejects an input reported as larger than 50 MiB. It then opens the URI to decode bounds only, computes a power-of-two `inSampleSize`, reopens the stream, and decodes a bitmap using `ARGB_8888`. The output format is selected by the user: JPEG at quality 80 or WebP at quality 75. On Android R and newer the WebP branch explicitly uses `WEBP_LOSSY`; on older versions it uses the older `WEBP` enum. Output is written into an app-cache directory, and the bitmap is recycled after the write. Android's API documentation defines `inSampleSize` as a request for subsampling, not an exact decoder-size guarantee [3]. Its `compress` method returns a boolean indicating success [4].

The method is an image *re-encoding* pipeline. It does not optimize the original compressed bitstream in place, preserve EXIF fields explicitly, or compare the encoded output with the source before reporting success. JPEG cannot retain alpha in the same way as a transparent source image [4]. The code ignores the boolean returned by `bitmap.compress`, then

uses the output file's length for its status message. This establishes a testable failure boundary: a completed flow is not, by itself, proof that an independently decoded file contains the intended pixels.

4.2 A wide-image counterexample to a hard dimension cap

The source names 4000 pixels as a maximum safe dimension. Its sample-size calculation, however, continues doubling only while *both* half-height divided by the current sample size is at least 4000 and half-width divided by the sample size is at least 4000. Consider a 10,000-by-1,000 image. The first condition on height is false, so the sample size stays one. A decoded `ARGB_8888` bitmap at full dimensions would occupy about 40 million bytes for pixels alone, before decoder and application overhead. This arithmetic trace follows the branch logic; it is not a measured peak process-memory figure. A 50 MiB compressed-file threshold does not bound decoded memory, because highly compressible images can expand greatly in memory.

The bound is also dependent on provider metadata. If the size query lacks `OpenableColumns.SIZE`, the method treats the size as zero and allows processing to continue. This may be reasonable for providers that omit length, but it means the 50 MiB check is not a universal input cap. A test corpus should pair compressed byte size with actual width, height, color model, and decoded allocation. A successful test on ordinary phone photos would not establish safety for a narrow panorama or an image with extreme dimensions.

4.3 Size accounting and visible claims

The summary divides both sizes by 1024 using integer arithmetic before computing a percentage. A small file can display zero kilobytes; a file that grows can yield a negative percentage while the English text still says "smaller." The format choice and fixed quality values also mean a photographic JPEG, a line-art PNG, and a transparent logo receive different kinds of loss. Image size reduction is therefore not a scalar property of the implementation. It depends on the input codec, resolution, content, chosen output format, and whether alpha or metadata matters to the user.

A defensible result requires at least four observations per input: original and output bytes, decoded dimensions, successful independent decode, and a content-fidelity judgment. Pixel error can be measured after aligning color spaces and accounting for orientation; a human check is needed for text, edges, and transparency that a simple global average can miss. For a screenshot containing small type, legibility may matter more than a large reduction in bytes. The present source contains no target-size search, no automatic quality adaptation, and no output verification. These absences are source observations, not evidence that the codec itself performs poorly.

5 PDF object rewriting: compression and semantic change

5.1 Path through the document

`PdfTools.compressPdf` queries a reported size and rejects files above 50 MiB. It initializes PDFBox-Android, opens the input stream, and loads the document using temporary-file-backed memory settings. It checks encryption, closes the document, and invokes a second method that reopens and reloads the same URI. The second load traverses COS objects. Image streams larger than 50 by 50 pixels can be decoded as Android bitmaps and re-encoded as JPEG for the three lossy modes. The selected mode changes maximum image dimension and JPEG quality; the UI call site chooses `HIGH`. A `dpi` field exists in the mode enum, but the inspected object-rewrite branch does not use it as a render resolution.

For a non-image stream without an existing Flate filter, the method reads the decoded bytes, deflates them at the library's best-compression level, removes old filter entries, and marks the new stream as Flate-decoded. Failures inside a single object are caught and skipped. This choice lets the loop continue when one object is problematic, but it also means a final saved file can contain a mixture of rewritten and untouched objects. The code saves the whole document and emits a success status without reopening it, rendering pages, or checking that every text layer, annotation, form, and embedded attachment survived.

Table 2: Source-level consequences of the PDF branches. A saved PDF still needs independent validation.

Object or mode	Implemented branch	Property not established by save alone
Lossy image stream	Bitmap decode and JPEG encode; dimensions and filter metadata reset.	Appearance, color, alpha, and page integrity.
Already Flate-filtered stream	No additional deflate step in this branch.	Whether bytes or semantics remain unchanged after save.
Other non-image stream	Decode and deflate when no Flate filter is found.	Parser compatibility and content preservation.
"Lossless" mode	Image JPEG branch skipped; metadata still cleared.	Metadata preservation or byte identity.

PDFBox documents saving as an operation on a parsed document, not as a guarantee of smaller output or equivalent rendering [5]. A PDF can grow after rewriting, particularly when its existing images or streams are already efficient. The code computes a reduction percentage only after saving. If no images were optimized and the reduction is not positive, it reports that the document is already at minimum size. That sentence is stronger than the evidence: the method has not searched all possible encodings or layouts, and a rewritten file may be larger for reasons unrelated to "minimum size." A future UI can report the observed byte counts without making an optimality claim.

5.2 The "lossless" label and metadata

The mode named `LOSSLESS` skips the JPEG image branch, but it still permits rewriting of non-image streams and clears catalog metadata and several document-information fields before saving. The producer field is changed to a legacy application string. As a result, the output is not byte-identical to the input and is not metadata-preserving. This does not automatically make the operation undesirable: metadata removal can be intentional. It does mean the word "lossless" needs a declared preservation criterion. Is the guarantee about visible pixels, extracted text, interactive fields, metadata, digital signatures, or every byte? The source implements no test demonstrating all of these.

An encrypted PDF is rejected before the rewrite path. This reduces one class of handling complexity but does not prove that every unencrypted PDF is safe to parse. A malformed object may trigger a caught per-stream exception, while a load or save exception becomes an error status. Resource limits also require measurement. Temporary-file-backed parsing can reduce pressure on heap, yet it can consume storage and still allocate decoded images. The comment "memory safe" should be treated as a design intention until peak RAM, native allocation, temporary disk, and failure rates are measured on devices.

6 Local processing and user-visible output

The file tools begin with user-selected URIs. Android's Storage Access Framework gives the application scoped access to a selected document [2]; it does not guarantee that a chosen provider is local. The source's compression methods read the stream and write their output to app cache. Later, the screen offers a save action that copies the output to a public Documents location. The save function has separate behavior for Android Q and newer and for older versions. This sequence is relevant to privacy and persistence: a success result in app cache is not the same event as a durable user-visible copy, and temporary files may remain until cache eviction or cleanup.

The screen also checks a usage quota before launching the picker and increments usage after a reported success. A quota gate is product behavior, not a codec property. It can change which files enter the method in a real usage study, so experimental tests should invoke the utility directly or document the tier and remaining quota. A failure path should not be silently excluded from denominator calculations because a picker was canceled, a provider lacked size metadata, or the save action failed. Each event has a distinct stage and a distinct user consequence.

On this source path there is no explicit HTTP call sending selected file bytes to a compressor service. The statement is deliberately local to the inspected methods. It does not cover billing, a cloud-backed document provider, analytics, a crash reporter, or another application feature. A falsifiable offline test would preload locally stored files, disable connectivity, monitor attempted network access, run each tool, and compare output hashes and file paths. The resulting trace must distinguish the application process from any provider process. No such trace accompanies this report.

7 Trace cases derived from the implementation

7.1 What a converter round trip can conceal

Suppose a test converts one pound to kilograms and then converts that result back to pounds. The use case multiplies by the enum factor on the first leg and divides by the same factor on the second. In exact arithmetic, the factors cancel even if the original factor was wrong. The round trip therefore tests arithmetic consistency but not conformity to the intended unit definition. This is a concrete reason to use an independent reference value. A test record should contain the source quantity, the reference factor with provenance, the internal unformatted result, and the displayed string. Without all four, a displayed value can appear accurate because of rounding while its underlying factor is stale.

Temperature has a related but distinct trap. A zero Celsius to Fahrenheit example tests the offset, and a hundred Celsius example tests the scale. Testing only zero would miss a wrong slope; testing only a nonzero point could hide an offset error. Kelvin examples must include an absolute-zero boundary and a value below it. The current formula accepts the latter as a number. An experiment should report separately whether the formula is mathematically reversible and whether the interface rejects an invalid physical state. Neither criterion should be inferred from the other.

The fixed-duration units provide another deliberately adversarial case. Convert one “month” to seconds under the enum: the answer is 2,592,000, exactly thirty 86,400-second days. Now compare a calendar interval from 1 February to 1 March in a non-leap year. Its elapsed days are not thirty. The code has no date input, so it cannot answer the second question; it is a quantity converter with a convention. This is not necessarily a defect if the convention is stated. It becomes a misleading result if a user interprets a calendar unit as an exact fixed duration. The same issue appears with binary data factors displayed under decimal-looking prefixes. The correct test is to ask what semantic contract the screen offers, then evaluate that contract.

7.2 The first failing image is not necessarily the largest file

A flat-color image can be stored in a comparatively small compressed file despite a very large raster. The method’s first gate inspects reported compressed bytes, while the costly stage allocates decoded pixels. For `ARGB_8888`, an unsampled raster requires roughly $4WH$ bytes of pixel storage for width W and height H. This estimate excludes decoder buffers, color conversion, object overhead, the output encoder, and other app activity; it is a lower bound for reasoning, not a measured resident set. A 12,000-by-900 image would require about 43.2 million pixel bytes if decoded at sample size one. Because the sampling loop requires both half-dimensions to exceed 4000, this wide fixture remains unsampled by the current rule. Its compressed file could be well under the 50 MiB metadata threshold.

Compare that case with an 8,000-by-8,000 image. Both half-dimensions equal 4000 at the first loop check, so the sample factor doubles. The two fixtures have different total pixel counts and aspect ratios; the important point is that the branch responds to the *smaller* dimension as well as the larger one. A paper claiming “maximum dimension 4000” would be falsified by the wide case. A future implementation might use a bound on $\max(W,H)$, on total decoded pixels, or both, but the present paper does not silently substitute such a design for the code being described.

Size reporting also needs an adversarial fixture. Feed the encoder a tiny source that rounds to zero KiB under integer division. The status may print a zero old size and a zero new size even when the two files differ in bytes. For a second fixture, choose an already efficient JPEG and re-encode it at quality 80. If the new file is larger, the calculated percentage becomes negative but the message template still appends “smaller.” These are statements about the formatting branches, not claims that a particular photo has been tested or that all JPEGs grow. The acceptance criterion is an output message that faithfully reflects the actual signed byte difference.

Finally, success must include more than existence of a path. The Android `Bitmap.compress` contract returns a success boolean, yet the method discards it. The file length is read afterward, but no independent decoder opens the bytes before a success event is emitted. A test harness can inject a failing output stream where feasible or use device-level fault simulation to check whether an encoder failure is reported correctly. That experiment is pending. The source-level issue is the missing check, not a documented incident of corrupt images in production.

7.3 PDFs can change without looking different

An input PDF with a populated title and author offers a minimal test for the mode called `LOSSLESS`. The image-recompression branch is skipped for that mode, but the document metadata fields are still cleared. A before-and-after metadata extraction would therefore show a change even if every rasterized page were identical. An input with a signature provides a more consequential case: an ordinary save that rewrites document bytes may invalidate a signature, but this

manuscript does not claim to have executed that case. A signature validator must decide the result; visual inspection of the page is insufficient.

The implementation's image path updates stream width, height, bit depth, color space, and filter fields after JPEG encoding. For an image originally using a different color space or transparency behavior, these assignments may affect appearance. The per-object catch block means a failed conversion is skipped rather than stopping the document. To understand a partially rewritten PDF, a test should record each object's initial filter, subtype, dimensions, output filter, and whether an exception was caught. The current method does not expose that ledger to the user. A final byte count cannot identify which objects were changed or why a document grew.

The "already optimized" message has a falsifiable counterexample too. A text-only PDF with no image objects can still contain inefficient non-image streams, embedded fonts, or other overhead. The source deflates certain streams, but it does not perform an exhaustive optimization search. If the saved file is not smaller, it may report that the file is already at minimum size because no images were optimized. That conclusion does not follow from the branch condition. The proper observation is only that the current rewrite yielded no measured reduction. This distinction matters when an optimization tool is compared with alternatives: a method's stopping point is not a theoretical minimum.

8 Evaluation design

8.1 Corpus and fixtures

The three operations need different fixtures but one provenance discipline. Every case should have a stable identifier, redistributable input or generator, expected behavior, and an explicit pass criterion. For numerical conversion, the matrix is category by source unit by target unit by magnitude and locale. Reference factors must be named and the unit convention stated. Cases near zero, sign boundaries, and display-rounding boundaries deserve their own rows. For temperature, include physically meaningful and deliberately invalid inputs to see whether the interface explains the distinction.

For image re-encoding, start with a natural photograph, line art, a text-heavy screenshot, a transparent image, and a wide panorama. Add a small file whose integer-kilobyte summary is coarse and a large-dimension file whose compressed byte size remains below the metadata cap. Inputs should record orientation, alpha channel, color profile, and source codec. Outputs should retain the exact encoded bytes for independent decode. If an input has private content, replace it with an author-created synthetic fixture; a paper cannot publish someone else's photograph merely to make the test reproducible.

For PDFs, include text-only, scanned, mixed text/image, already optimized, metadata-rich, annotated, form-bearing, and signed or encrypted examples where rights permit. A signed document is particularly informative because any rewrite can invalidate a signature even if pages look identical. Each file needs a predeclared preservation target: visible pages, extractable text, forms, annotations, bookmarks, metadata, signatures, or a stated subset. Some inputs may legitimately be rejected. Those rejections should be recorded, not deleted from the test set after inspection.

Table 3: Proposed measurements; no numerical results are available for the inspected build.

Path	Primary measures	Independent check
Conversion	Error in base units; formatted-string error; round-trip drift.	Standards-based factors and locale-specific parse/display test.
Images	Byte ratio, dimensions, decode success, visual and alpha fidelity.	Second decoder plus aligned rendered comparison.
PDFs	Byte ratio, page count, text, metadata, forms, annotations, rendering.	Independent parser and page rasterizer.
All paths	Completion, latency, peak memory, temporary storage, network attempts.	Physical-device runs and process-level instrumentation.

8.2 Accounting and analysis rules

For a file with input bytes B_{in} and output bytes B_{out}, define a signed size change

$$R = 1 - \frac{B_{\text{out}}}{B_{\text{in}}}, \qquad B_{\text{in}} > 0. \tag{2}$$

A negative R means the file grew. Report bytes as well as the ratio so a large percentage on a tiny file is not mistaken for substantial storage savings. A codec may produce a smaller file by discarding unacceptable content; therefore size reduction is not an optimization result unless the declared preservation check passes. For numerical conversions, report absolute error in the target unit and a relative error only where the reference value is meaningfully nonzero. Display rounding belongs in a separate column.

Timing starts when the method is invoked with a readable URI and ends when a complete output file exists. The picker and public-save intervals can be reported separately because provider latency and user action are outside the transformation. Cold and warm runs should be distinguished. Named devices need Android version, chipset, RAM, free storage, and build identifier. Record failures and timeouts alongside successful runtimes; excluding them would make the tail look artificially fast. A memory claim needs peak process allocation and temporary-disk use, not only Java heap. A zero network attempt claim needs the exact monitoring scope and period.

For PDF and image fidelity, the result table should carry feature-specific failures. A transparent icon flattened to an opaque JPEG, a PDF whose metadata was stripped, and a PDF with missing text are different outcomes. A binary "opened" field is too weak; a forgiving viewer may display a partly damaged PDF. Independent parsers, extraction tools, and rasterized page comparisons reduce the chance that producer and validator share the same defect. Human review of a stratified sample can catch visual defects that the byte checks miss. This protocol describes work to be done, not completed results.

9 Proposed execution protocol

9.1 A manifest before opening the file

Each fixture needs a manifest written before the app is run. For unit conversion, it lists the category, source and target units, quantity, reference value, expected display locale, and allowed tolerance. For an image, it lists dimensions, input codec, alpha presence, orientation, expected textual regions, and the intended preservation requirement. For a PDF, it lists page count, extractable text by page, annotations, form fields, metadata fields, and signature status. The manifest is an oracle for the experiment, not a file generated from the output. Creating it afterward would allow a defect to be normalized into the expected result.

The validator should save three kinds of evidence for each file transformation: the original bytes, the output bytes, and a machine-readable trace of what was checked. Hashes help ensure that the input and output being reviewed are the same files used for measurement; they do not prove semantic equivalence. A byte-identical output is easy to classify, but a changed output requires format-specific inspection. For an image, that means decoding it and checking dimensions, orientation, alpha, and a representative render. For a PDF, it means parsing in a second implementation, extracting text and document properties, and comparing page images at a fixed rasterization setting. The source's own success event is recorded as one observation, not treated as the independent validator.

Fixture selection should prevent easy cases from dominating the report. A dozen nearly identical photographs cannot stand in for screenshots, transparent logos, and wide panoramas. Similarly, dozens of one-page PDFs with no annotations do not test forms or metadata preservation. The corpus can be small at first, but each file's role must be explicit. A single file may carry several features, yet failure attribution is easier when a deliberately minimal fixture isolates one feature. The paper should publish the complete list of attempted cases, including those that fail to parse or that the UI rejects. Otherwise an apparently high success rate can be manufactured by omission.

9.2 Device runs and failure accounting

An on-device test session should identify the installed build, Android version, device model, available RAM, free storage, and whether the input originated in local storage or a cloud-backed provider. Record the time spent obtaining the URI separately from transformation time. For image and PDF operations, perform cold and warm runs with the same bytes and a documented order. If one run fails, keep the failure in the result file rather than replacing it with another attempt. Repeated runs of the same file characterize variability; they do not increase the number of independent file examples.

The output stage has two transitions: creation in app cache and copying to public storage after user action. A complete user-workflow test checks both and records the path or URI returned at each stage. If the cache file is valid but the save step fails, that is a user-visible failure even though the algorithm finished. If a provider gives no size metadata, record that fact because the 50 MiB gate cannot be interpreted normally. If a PDF object is skipped after an exception, record the event and the final validation status. The current code may not expose every per-object skip, so instrumentation would be needed for a full trace; this is a design requirement for a future experiment, not a result of the present audit.

Resource observations should include peak resident memory, Java heap, native allocation where available, and temporary disk. A compressed-byte limit cannot substitute for those measures. Run order can influence memory pressure and thermal throttling, so alternate or randomize fixture order and report it. A fixed timeout makes hangs countable. For a security-oriented stress set, begin with synthetic extreme dimensions and structurally unusual PDFs rather than private user documents. A research paper must not imply that its input corpus is safe simply because a parser accepted it once.

An offline test needs its own setup log. Preload the inputs locally, disable network access, start process-level traffic observation, run each tool, and record attempted connections during the operation. A separate run can investigate provider behavior. The result should say "no observed attempt from process X during operation Y under setup Z" if that is what the trace shows. It should not become an absolute statement that the entire application never uses a network. No traffic capture was performed for this draft.

9.3 Interpreting a small result set

With a small convenience corpus, report per-fixture outcomes and failure traces rather than a percentage that suggests population-level generality. An image-compression ratio without a legibility check, or a PDF size reduction without a form check, is an incomplete result. A later study with a larger permission-cleared corpus could report distributions, confidence intervals by file family, and stratification by content type. The initial goal is more modest: determine whether the current source behavior matches its own UI claims, and identify the smallest changes needed for truthful status reporting and reliable validation.

The unit converter can be assessed separately with generated cases because its state space is compact and deterministic. The PDF and image paths need actual files and device observations. Combining their outcomes into one "Logic Engine accuracy" number would erase this methodological difference. A useful report can still place the three paths side by side as parts of one user-facing feature, but it must preserve each path's denominator, acceptance criterion, and failure type. This is the central reporting rule for a heterogeneous utility interface.

10 Validity, security, and release limits

Static inspection can confidently identify the constants, branches, and missing explicit checks in this source snapshot. It cannot prove that a given Android device decodes a particular file the same way as another, or that a provider reports the same size metadata. PDFBox-Android is a fork with its own version and behavior; a reference to desktop PDFBox documentation explains the class of operation but cannot substitute for testing the Android dependency actually shipped. Likewise, Android's bitmap API documents a contract, while codec implementation details vary by platform release. An empirical version must pin these versions and keep failed files as evidence.

Input parsers also form a security boundary. Oversized decoded images, malformed PDFs, object graphs with many streams, and crafted provider URIs can exhaust time, memory, or temporary storage. The observed source has a reported-byte gate and temp-backed PDF parsing, but neither is a complete bound on decoded work. The study has not performed adversarial fuzzing or a security audit. Public code or example files should be selected by an allowlist and checked for credentials, private documents, and internal endpoints before any release. This manuscript itself contains no API key or private endpoint.

The author should distinguish a product release from a research artifact. A research package can include the source revision, build instructions, license boundaries, synthetic fixtures, expected-output manifests, and scripts for independent checks. The Android app may remain branded Vanexa in its existing distribution channel while the author's research identity is Causyne. That difference is a naming fact, not a claim that application code was renamed. No public release, DOI, ORCID verification, or peer review is asserted here.

11 Traceability for an empirical revision

A future reader needs to know which source method supports each statement. The converter's linear factors are in the unit enums, while the affine formulas are in `UnitConverterUseCase`. Parsing and six-decimal display belong to the converter view model, a different layer. The image path's 50 MiB metadata gate, sample-size loop, output format, and quality constants are all in `ImageTools`. PDF object treatment is in `PdfTools`; user selection of `HIGH` and the separate save action are in `PdfToolScreen`. These locations matter because a UI change can alter the tested mode without changing the PDF optimizer, and a unit-enum change can alter numerical behavior without changing the view model.

Table 4: Traceability map for the principal source-level findings. The filenames identify the inspected Android implementation, not a public release archive.

Finding	Source boundary	Minimal independent check
Fixed conversion factors and durations	Unit enums; `UnitConverterUseCase`	Compare against declared standards and conventions.
Six-place display and locale parsing	`UnitConverterViewModel`	Set locales and round-trip displayed strings.
Wide image may remain unsampled	`ImageTools` sample-size helper	Feed known dimensions and inspect decoded allocation.
Encoder return value is ignored	`ImageTools.compressImage`	Force or simulate encoder failure; decode output.
PDF mode strips metadata	`PdfTools.compressObjectLevel`	Extract document properties before and after.
UI invokes `HIGH` mode	`PdfToolScreen` picker callback	Record selected mode during a device run.

This map is also a safeguard against over-attribution. A class comment promises memory safety, but the actual code only shows particular measures: a reported file-size gate, temporary-file-backed PDF parsing, and image subsampling under a two-dimension condition. A manuscript cannot turn those measures into an observed memory bound without measuring the device process. Similarly, the phrase "offline compression" can truthfully describe the absence of an explicit upload in the inspected compressor methods, while an end-to-end privacy assertion would require provider provenance and a network observation. The evidence required for each claim should be written next to the claim before publication, not reconstructed after a reviewer asks for it.

For reproducibility, a later public source snapshot should identify a revision and include only files the author is entitled to distribute. The revision used for a benchmark must match the binary installed on the phone. Device logs, fixture manifests, and validation outputs should refer to that revision. If the source changes while experiments are running, results from the earlier build belong in a separate group. This discipline prevents a familiar error in engineering papers: describing the newest code while reporting measurements made against an older executable.

12 Conclusion

The current utility interface combines three legitimate local tasks under one entry point. Their implementations, however, offer different kinds of guarantees. The converter computes with fixed factors and displays six-decimal strings. The image tool re-encodes a sampled bitmap at a fixed quality; its compressed-file gate is not a decoded-memory bound. The PDF tool rewrites objects and metadata; its mode named lossless does not preserve every document property. These are traceable findings about source behavior, not estimates of real-world failure frequency.

The next step is a small, permission-cleared fixture set with independent validators and physical-device measurements. Results should show reductions and regressions together, including files that grow, images that lose useful detail, and PDFs whose nonvisual properties change. Such evidence would make it possible to say where the tools help users, which inputs require a warning or abstention, and which guarantees the application can honestly make.

References

[1] A. Thompson and B. N. Taylor. *Guide for the Use of the International System of Units (SI)*. NIST Special Publication 811, 2008. NIST online edition.

[2] Android Developers. *Access documents and other files from shared storage*. Android documentation.

[3] Android Developers. *BitmapFactory.Options API reference*. Android API reference.

[4] Android Developers. *Bitmap API reference*. Android API reference.

[5] Apache Software Foundation. *PDFBox 2.0 PDDocument API reference*. PDFBox API reference.

Copyright 2026 Ikbal Fadilah. This manuscript is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).